UNIVERSIDAD NACIONAL DE COSTA RICA

DE LA SEDE NICOYA

Curso: Programación 1 – EIF201

Proyecto: Implementación de Juego Blackjack por Consola

Estudiantes:

Daniela Venegas Zuñiga – 208440873

Sasha Castillo Montiel - 208770716

Krisaneth Lopez Gomez - 504600973

Profesor: Francisco Coulon Olliver

Fecha de Entrega: 30 de octubre de 2026

1. Introducción

El desarrollo de software orientado a objetos constituye una de las metodologías fundamentales en la ingeniería de software actual, permitiendo modelar sistemas del mundo real mediante la abstracción, el encapsulamiento y modularidad. El presente documento detalla la primera fase del diseño y desarrollo de un sistema de simulación del juego de cartas Blackjack por consola, implementado en el lenguaje de programación C++.

A lo largo de este informe se establece el diseño estructural del sistema mediante diagramas de clases UML, la definición formal de las reglas del negocio, los casos de prueba prácticos y el avance de código correspondiente a las estructuras base del programa. Este proyecto no solo busca replicar las reglas lógicas y de apuestas de un casino tradicional, sino también aplicar buenas prácticas de control de versiones y trabajo colaborativo en equipo.

2. Definición del problema

El problema consiste en diseñar e implementar un sistema de software autónomo ejecutado por consola que permita simular de forma fiel y estructurada una partida de Blackjack. El sistema debe ser capaz de gestionar múltiples reglas de negocio complejas de manera automatizada y sin intervención de errores humanos.

Entre los retos principales que el sistema debe resolver se encuentran:

- **Gestión de múltiples mazos:** Administrar un zapato compuesto por 6 barajas estándar (312 cartas en total), garantizando que se barajen una única vez al inicio y controlando su agotamiento de manera estricta.
- **Control de estados y valores dinámicos:** Interpretar correctamente el valor flexible del As (1 u 11 puntos) de forma dinámica según la puntuación actual de la mano para evitar pasarse de 21.
- **Manejo financiero y restricciones de saldo:** Controlar un fondo inicial de fichas por jugador, validando en tiempo real si el usuario cuenta con el saldo suficiente para realizar apuestas, doblar la apuesta o dividir las manos (*split*).
- **Flujo de turnos condicionales:** Automatizar las reglas estrictas de la casa (el crupier debe pedir cartas obligatoriamente hasta sumar 17 o más) y resolver situaciones especiales como el seguro, los empates (*push*), las derrotas por pasarse de 21 (*bust*) y los pagos con redondeo hacia abajo en caso de decimales.

3. Reglas del juego y ejemplos de uso

Resumen de reglas clave

El Blackjack se rige bajo las siguientes normativas principales:

- **Objetivo:** Obtener una mano con un valor superior a la del crupier sin superar los 21 puntos.
- **Configuración:** Se emplean 6 mazos de 52 cartas (312 en total) que se barajan una única vez al inicio de la partida.
- **Valor de las cartas:** Las cartas del 2 al 10 valen su valor numérico; las figuras (J, Q, K) valen 10 puntos; y el As (A) vale 1 u 11 puntos de manera flexible según convenga para evitar pasarse de 21.
- **Blackjack:** Ocurre cuando las primeras dos cartas suman exactamente 21 puntos. Paga 3:2 si el crupier no lo tiene.
- **Acciones del jugador:** En su turno, el jugador puede *pedir* carta, *plantarse*, *doblar* su apuesta (recibiendo una única carta extra) o dividir (*split*) si sus dos cartas iniciales tienen el mismo valor.
- **Turno del crupier:** La casa debe pedir cartas obligatoriamente hasta alcanzar 17 puntos o más. Si el crupier se pasa de 21, ganan todos los jugadores vigentes.

Ejemplo 1: Victoria estándar (Pago 1:1)

- **Cartas iniciales del jugador:** [9,8] — Total: 17 puntos. El jugador decide *plantarse*.
- **Cartas finales del crupier:** [10,6] — Total: 16 puntos. El crupier está obligado a pedir, saca un 5 y suma 21 puntos. (Nota: En este caso el crupier se pasa o suma más, hagamos un ejemplo clásico de victoria por puntaje mayor)
- **Cartas finales del crupier alternas:** [10,6] — Total: 16 puntos. Pide carta, saca un 8 y se pasa con 24 puntos.
- **Resultado:** El jugador no se pasó y el crupier superó los 21 puntos. El jugador gana de forma automática.
- **Apuesta y Pago:** Apuesta inicial de ₡100. El jugador gana y recibe un pago de 1:1, obteniendo ₡100 de ganancia neta (recupera sus ₡100 apostados + ₡100 de premio).

Ejemplo 2: Derrota por pasarse de 21 (*Bust*)

- **Cartas iniciales del jugador:** [10,5] — Total: 15 puntos. El jugador decide *pedir* una carta adicional y recibe una J (valor 10).
- **Total de la mano:** $10 + 5 + 10 = 25$ puntos.
- **Resultado:** El jugador supera el límite permitido de 21 puntos, por lo que pierde la ronda de inmediato, independientemente de lo que haga el crupier posteriormente.
- **Apuesta y Pago:** Apuesta inicial de ₡50. El jugador pierde su apuesta, la cual es retenida por la casa.

Ejemplo 3: Empate (*Push*)

- **Cartas finales del jugador:** [10,8] — Total: 18 puntos (el jugador se plantó).
- **Cartas finales del crupier:** [9,9] — Total: 18 puntos (el crupier se plantó al llegar a 17 o más).
- **Resultado:** Ambas manos obtienen exactamente el mismo puntaje sin tratarse de un blackjack. Se declara un empate técnico.
- **Apuesta y Pago:** Apuesta inicial de ₡200. Se produce un empate y se devuelve la apuesta íntegra al jugador (₡200), sin ganancias ni pérdidas.

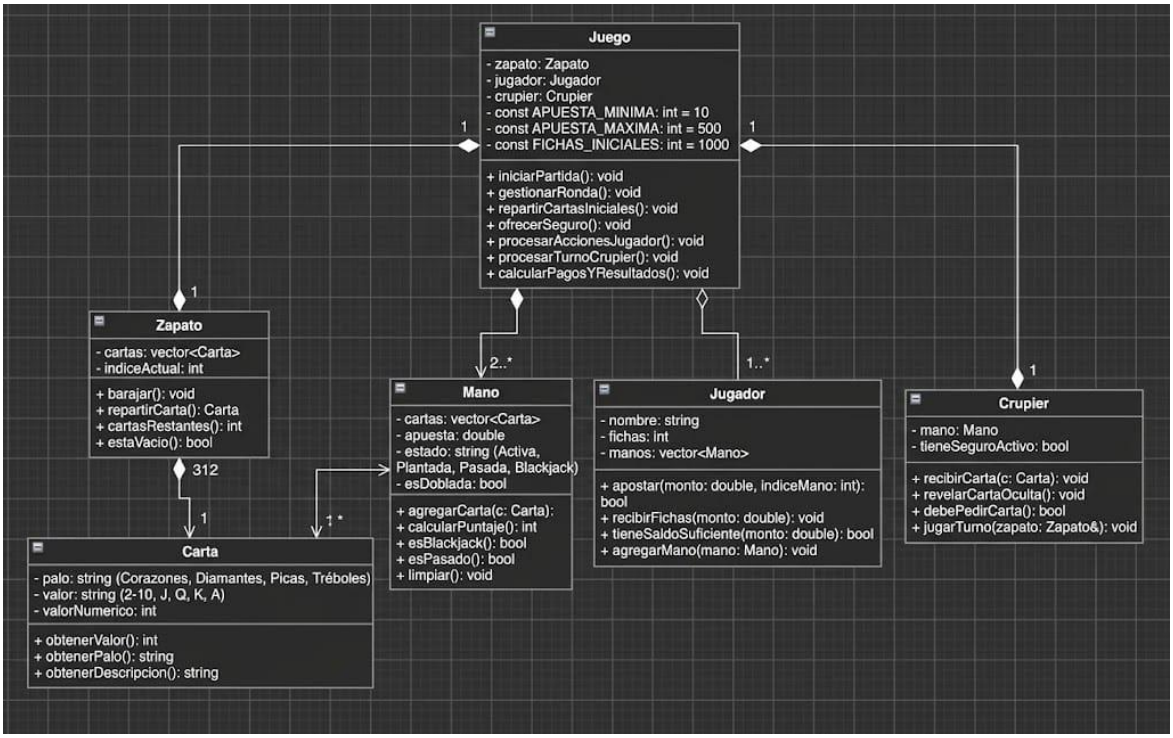
Ejemplo 4: Blackjack natural (Pago 3:2)

- **Cartas iniciales del jugador:** $[As, K]$ — Total: 21 puntos exactos con las primeras dos cartas.
- **Cartas iniciales del crupier:** Su carta visible es un 6 y su oculta un 8 (Total: 14, pide y llega a 19). El crupier no tiene Blackjack.
- **Resultado:** El jugador obtiene un Blackjack natural y el crupier no.
- **Apuesta y Pago:** Apuesta inicial de ₡100. Al tratarse de un blackjack, se aplica el pago de 3:2. El jugador recibe ₡150 de ganancia neta más su apuesta original (total de devolución: ₡250).

Ejemplo 5: Caso de Dividir (*Split*)

- **Cartas iniciales del jugador:** [8,8] — Total: 16 puntos con un par del mismo valor. El jugador decide dividir.
- **Desarrollo:** Separa sus cartas en dos manos independientes e injecta una segunda apuesta igual a la original (₡100 adicionales).
    - Mano 1: Recibe un 5 [8,5] (Total: 13), pide carta y recibe un 7 (Total: 20). Se planta con 20.
    - Mano 2: Recibe un As [8, A] (Total: 19). Se planta con 19. (Nota estricta del reglamento: un 21 obtenido tras dividir paga 1:1, no blackjack).
- **Cartas finales del crupier:** 18 puntos.
- **Resultado:** Ambas manos del jugador (20 y 19) vencen los 18 puntos del crupier.
- **Apuesta y Pago:** Dos apuestas de ₡100 cada una (Total arriesgado: ₡200). Ambas manos ganan con pago 1:1, generando una ganancia total de ₡200 netos.

4. Diagrama de clases UML y descripción de responsabilidades



Descripción de responsabilidades a alto nivel:

- **Carta:** Encapsula la identidad fundamental del naipe (palo, texto visible y valor numérico base para los cálculos matemáticos).
- **Zapato:** Actúa como el contenedor masivo de los 6 mazos (312 cartas), asegurando que el barajado ocurra una única vez al inicio y controlando el índice de reparto hasta agotar existencias.
- **Mano:** Contiene la lógica algorítmica crítica de la partida, evaluando de forma dinámica el valor flexible del As (1 u 11), detectando Blackjacks naturales y controlando los límites del puntaje.
- **Jugador:** Administra el estado financiero del usuario (fichas iniciales, validación de saldos para apostar, doblar y asegurar) y soporta múltiples manos en caso de división.
- **Crupier:** Modela el comportamiento automatizado de la casa, rigiéndose estrictamente por la regla de pedir cartas hasta alcanzar 17 o más y gestionando la revelación de la carta oculta.
- **Juego:** Funciona como el controlador central (*Controller*) que coordina el ciclo de vida global de la partida, el orden de los turnos, las reglas de apuestas mínimas y máximas, y la liquidación de pagos.

5. Plan de acción del proyecto

(Lógica y Estructura POO):

- Diseño e implementación de las clases base del sistema (Carta, Zapato, Mano).
- Desarrollo de la lógica matemática para el cálculo dinámico de los Ases (1 u 11) y el control del zapato de 6 mazos.
- Pruebas unitarias de consola para la Fase 1.

(Arquitectura, Control y Documentación):

- Diseño de las clases de control e interacción (Jugador, Crupier, Juego).
- Creación y refinamiento del diagrama de clases UML y redacción de la justificación a alto nivel de cada componente.
- Elaboración del documento formal en PDF (introducción, definición del problema, redacción de las reglas y los 5 ejemplos de uso con moneda local).

- Configuración del repositorio Git y control de versiones del proyecto.

- **Cronograma de actividades**

  Fase 1: Análisis, Diseño UML y Avance Base (Semana 18 al 30 de octubre de 2026)

  - Actividad 1: Definición formal de las reglas del negocio y casos de uso prácticos.
  - Actividad 2: Modelado del diagrama de clases UML y especificación de multiplicidades.
  - Actividad 3: Implementación en C++ de la estructura modular (.h y .cpp) para cartas, zapato y manos.
  - Actividad 4: Integración de la documentación formal en PDF y preparación de la exposición.

  Fase 2: Implementación de la Lógica de Juego y Turnos

  (Próxima fase)

- **Minutas de el trabajo**

Minuta #1: Planificación y Análisis de Requerimientos

Fecha: 18 al 19 de octubre de 2026

Hora: 02:00 p.m. - 04:15 p.m.

Temas trabajados:

- Análisis detallado de la rúbrica de la Fase 1.
- Estudio sobre las reglas del Blackjack (manejo de los 6 mazos, límites de apuestas en colones y el comportamiento del As).
- Definición de la estructura modular en archivos .h y .cpp para cumplir con los principios de Programación Orientada a Objetos.

Próxima trabajo: 21 de octubre para la revisión de avance

Minuta #2: Integración del Diseño UML y Pruebas de Código

Fecha: 20 al 21 de octubre de 2026

Hora: 01:00 p.m. - 04:00 p.m.

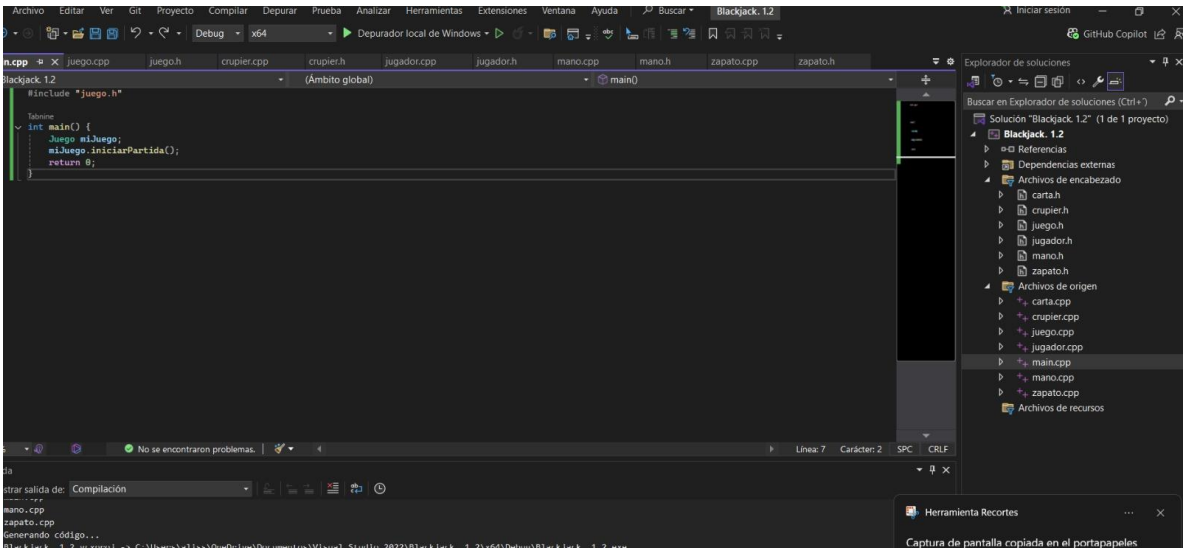
Temas trabajado:

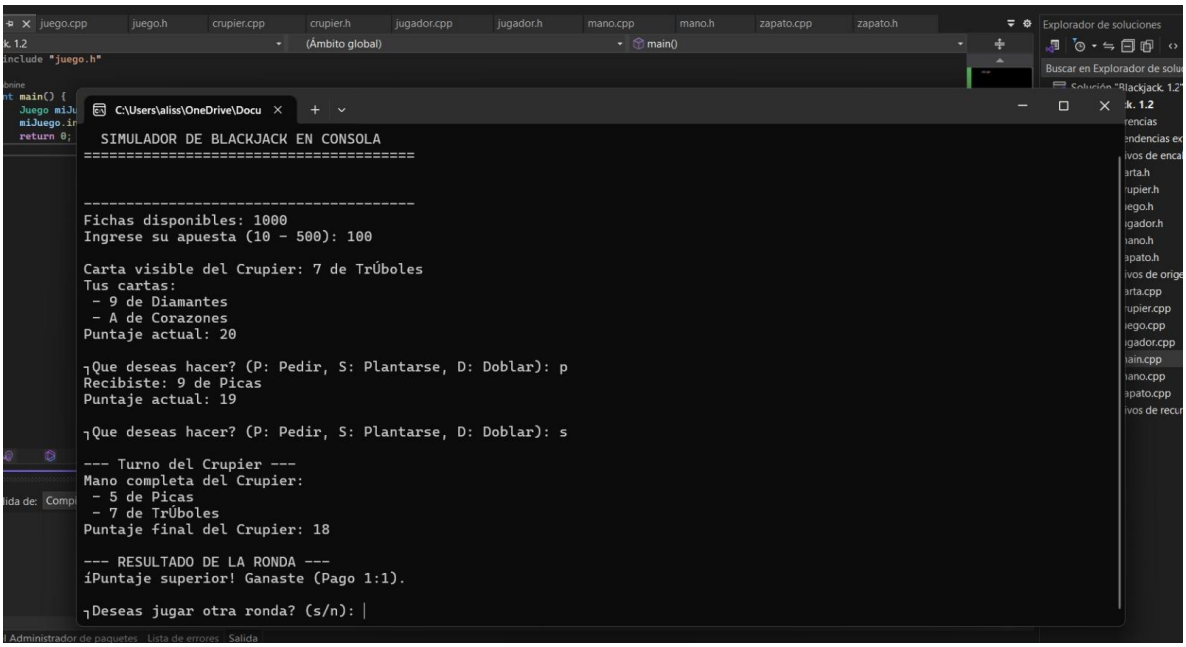
- Revisión conjunta del diagrama de clases UML y validación de las multiplicidades (relaciones entre Juego, Zapato, Jugador y Crupier).
- Pruebas de compilación del código base en C++ (verificación del barajado único de 312 cartas y reparto inicial).
- Unificar la documentación final en formato PDF e incluir las capturas de pantalla de la consola.

Próximo trabajo: 25 de octubre para preparación de la exposición.

6. Avance de la implementación

- **Descripción del estado actual del código:** Se han programado las clases estructurales base (Carta, Zapato, Mano, Jugador, Crupier y Juego), logrando compilar exitosamente en C++. Actualmente el sistema es capaz de inicializar el zapato con 312 cartas, ejecutar el barajado único, repartir las cartas iniciales y calcular dinámicamente el valor de una mano considerando la flexibilidad del As.




juego.cpp
juego.h
crupier.cpp
crupier.h
jugador.cpp
jugador.h
mano.cpp
mano.h
zapato.cpp
zapato.h
(Ámbito global)
main()
Explorador de soluciones
C:\Users\aliss\OneDrive\Docu
SIMULADOR DE BLACKJACK EN CONSOLA
Fichas disponibles: 1000
Ingrese su apuesta (10 - 500): 100
Carta visible del Crupier: 7 de TrÚboles
Tus cartas:
- 9 de Diamantes
- A de Corazones
Puntaje actual: 20
¬Que deseas hacer? (P: Pedir, S: Plantarse, D: Doblar): p
Recibiste: 9 de Picas
Puntaje actual: 19
¬Que deseas hacer? (P: Pedir, S: Plantarse, D: Doblar): s
--- Turno del Crupier ---
Mano completa del Crupier:
- 5 de Picas
- 7 de TrÚboles
Puntaje final del Crupier: 18
--- RESULTADO DE LA RONDA ---
íPuntaje superior! Ganaste (Pago 1:1).
¬Deseas jugar otra ronda? (s/n):